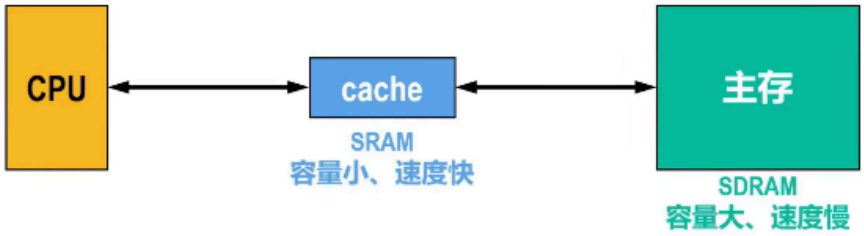
一、作用

cache的相关基本概念 —— cache的作用

- 主存一般采用容量大、功耗较小、成本较低的同步动态随机存取存储器SDRAM（目前主流为DDR4或DDR5）。
- 静态随机存取存储器SRAM的容量小、功耗大、成本高，但SRAM的访问速度远高于SDRAM。
- 因此，为了提升CPU访问主存的性能，通常会在CPU与主存之间添加一个SRAM作为高速缓冲存储器cache。
  - 将主存中经常访问或即将访问的数据，复制一份（调度）到cache中，使得大部分数据访问都可以在cache中进行，从而提升系统性能。
  - 采用这种方法的主要原因是CPU执行的程序具有较强的程序局部性。



二、程序局部性

cache的相关基本概念 —— 程序局部性

- 程序局部性是指，在一段时间内，整个程序的执行仅限于一个较小的局部范围内。具体来说，程序的局部性又表现为以下两种：
  - 时间局部性：若程序在某个时刻访问了一个存储位置，该位置在未来可能会被多次访问。
    - 例如程序中的循环结构和调用过程就很好地体现了时间局部性。
  - 空间局部性：若程序访问了某个存储位置，则其附近的存储位置也可能被访问。
    - 例如程序中的数组、结构体成员、顺序执行的代码块，通常在主存中是按顺序存放的，对它们的访问，就具有较强的空间局部性。

【举例】请尝试分析以下C语言程序代码的程序局部性。

```
for (i=0; i<1000; i++)  sum+=a[i];
```

分析

for循环中的指令序列（读取、加法）在主存中顺序存放，具有空间局部性；

for循环中的指令序列被执行1000次，具有时间局部性；

变量i、sum是单个变量，不存在空间局部性；由于其在每次执行for循环时都会被用到，因此具有时间局部性。

数组a包含至少1000个数据元素，这些数据在主存中顺序存放，具有空间局部性；数组a中的每个数据元素都只被访问一次，因此不存在时间局部性。

例题

【2017年 题14】某C语言程序段如下：

```
for (i=0; i<=9; i++) {
    temp = 1;
    for (j=0; j<=i; j++)  temp *=  a[j];
    sum += temp;
}
```

下列关于数组a的访问局部性的描述中，正确的是（A）。

A. 时间局部性和空间局部性皆有

B. 无时间局部性，有空间局部性

C. 有时间局部性，无空间局部性

D. 时间局部性和空间局部性皆无

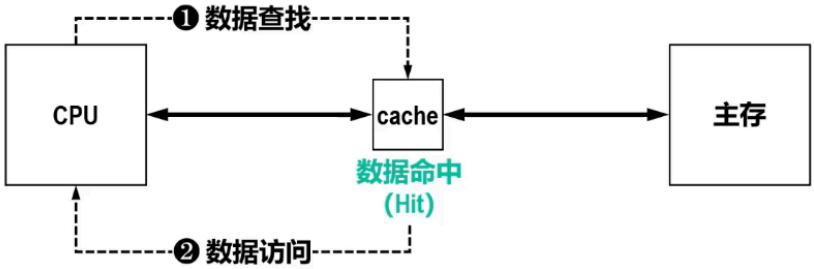
解析

数组a中的各元素在主存中顺序存放，具有空间局部性；
数组a的读取序列：a[0]、a[0]、a[1]、a[0]、a[1]、a[2]、…，
数组a中的同一元素被多次访问，具有时间局部性。

三、访问时间

3.1 命中缓存时

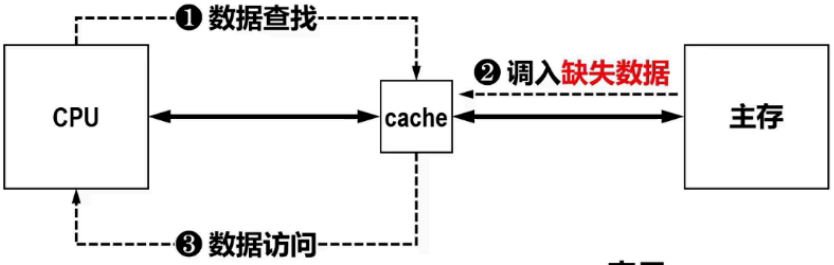
■ 在CPU和主存之间添加了cache后，CPU不再直接访问慢速的主存，而是通过字节地址访问快速的cache。



上述情况下，数据访问时间称为命中访问时间，记为t_c { cache内的查找时间；cache访问时间

3.2 未命中缓存时（数据缺失）

■ 为了便于快速查找，主存和cache都被划分成若干个固定大小的数据块（Block），每个数据块又包含若干个字。



表示

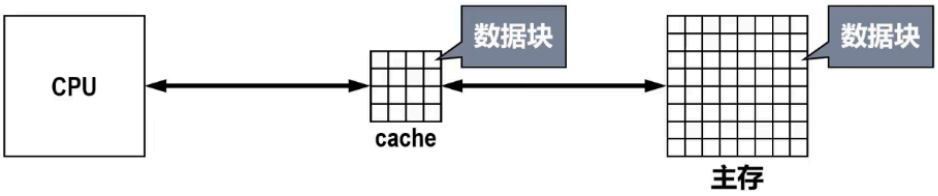
上述情况下，数据访问时间称为缺失补偿（Miss Penalty）
- cache内的查找时间
- 主存访问时间（较为漫长），记为t_m
- cache访问时间

四、数据分块

■ 为了便于快速查找，主存和cache都被划分成若干个固定大小的数据块（Block），每个数据块又包含若干个字。

□ 若进行数据访问时出现数据缺失的情况，则需要将缺失数据所在的数据块从慢速主存载入cache中。

□ 因此，缺失数据相邻的数据也会随着数据块一起载入cache。

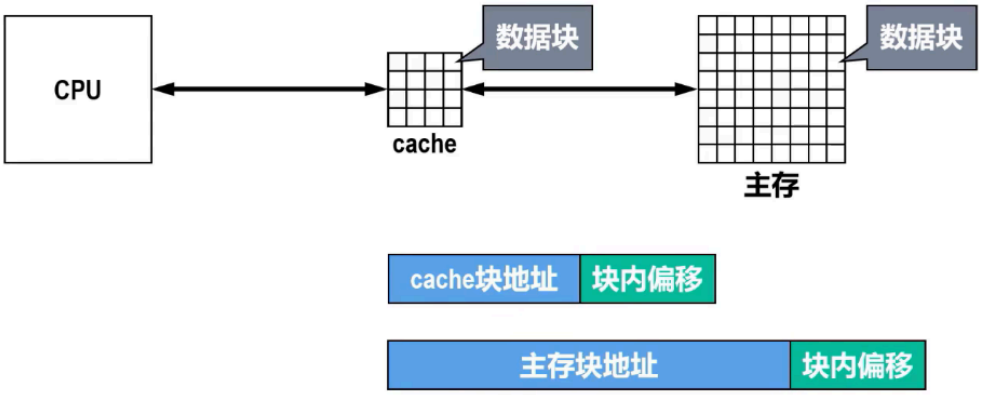


- 上述预读策略可以充分利用程序的空间局部性，提高顺序访问的命中率。
- 然而，数据块的大小对cache有较大影响：
  - 数据块过小：无法利用预读策略优化程序的空间局部性。
  - 数据块过大：将使得替换算法无法充分利用程序的时间局部性。

■ 进行数据分块后，需要给主存内的数据块和cache内的数据块分配地址。

☐ 数据块的地址由块地址和块内偏移地址（简称块内偏移Offset）两部分构成。

☐ 由于cache容量比主存容量小很多，因此cache内的块地址长度小于主存内的块地址长度。



五、评价指标

某程序运行期间命中cache的次数记为 n_c，从主存中访问信息的次数记为n_m，命中率（Hit Ratio）记为h：

$$h = \frac{n_c}{n_c + n_m}$$

h越接近于1性能越好

$1-h$ 称为缺失率（Miss Ratio）。命中情况下的访问时间记为t_c，数据缺失情况下的访问时间记为t_m，cache/主存系统的平均访问时间记为t_a：

$$t_a = ht_c + (1-h)t_m$$

h越接近于1，t_a越接近t_c

访问效率记为e：

$$e = \frac{t_c}{t_a} = \frac{t_c}{ht_c + (1-h)t_m} = \frac{1}{h + (1-h)\frac{t_m}{t_c}} = \frac{1}{h + (1-h)r}$$

h越接近于1，e越接近1

r一般为5~10，不能太大